

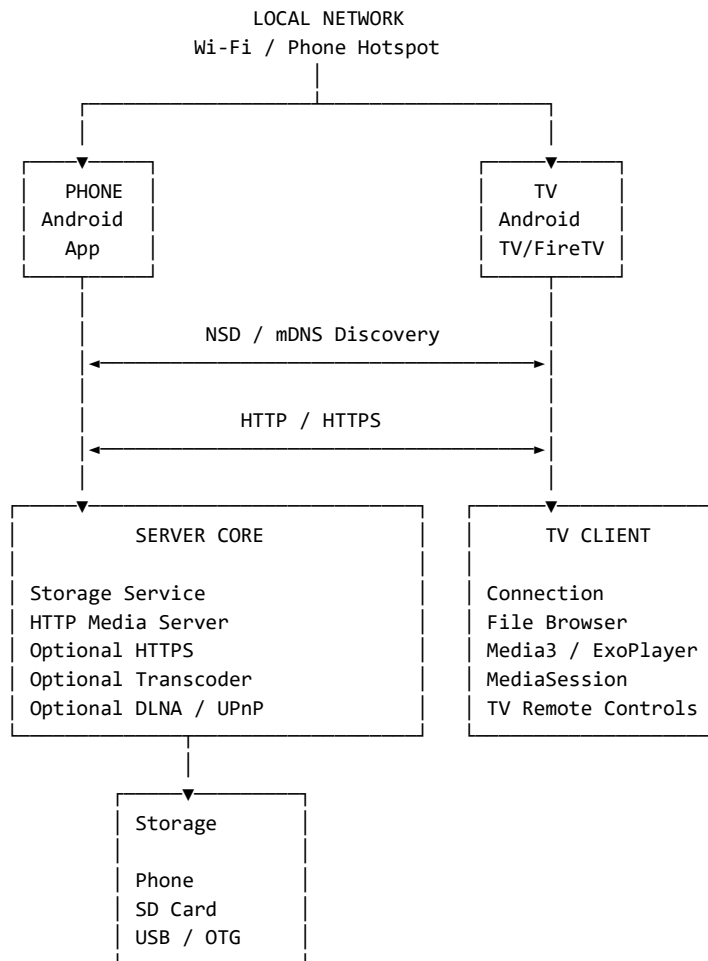
LocalStream — Architecture & Build Guide

Local-only media streaming from Android phone / USB to Android TV / Fire TV

1. Product Goal

A two-app system where an Android phone temporarily acts as a local media server. The phone shares a selected folder over Wi-Fi or a phone hotspot. A dedicated Android TV / Fire TV client discovers the phone, browses media, and plays files directly without internet, cloud upload, Google Cast, or screen mirroring.

2. High-Level Architecture



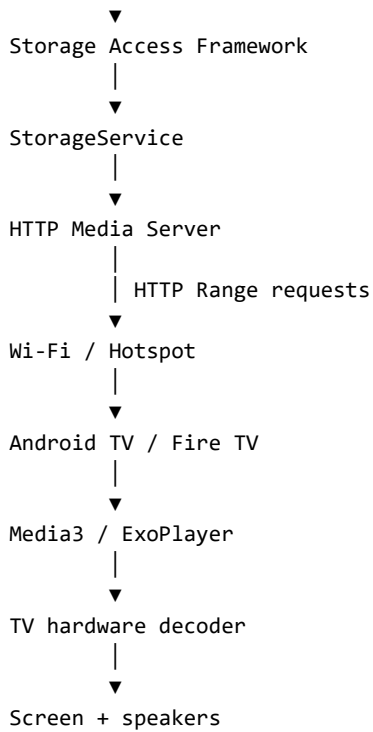
Optional compatibility path:

Phone — DLNA / UPnP → Other DLNA TVs / Players

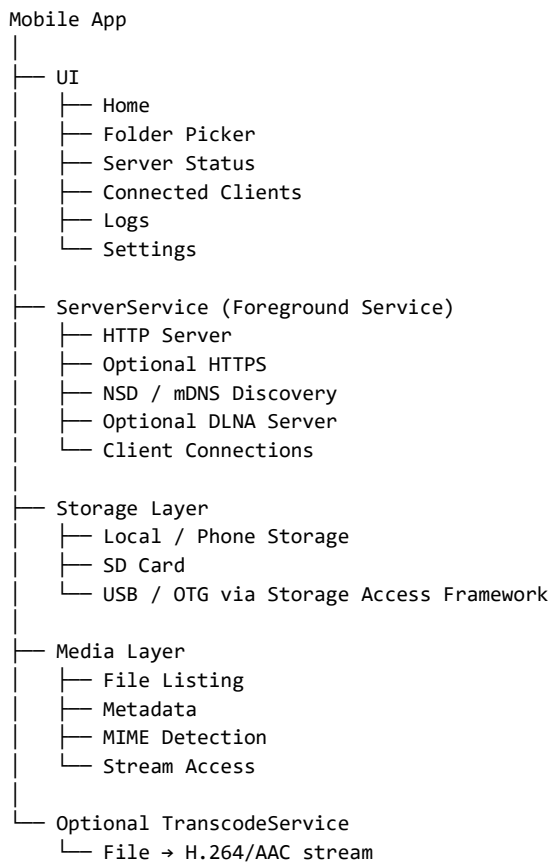
3. Core Data Flow

Direct playback is the default and preferred path:

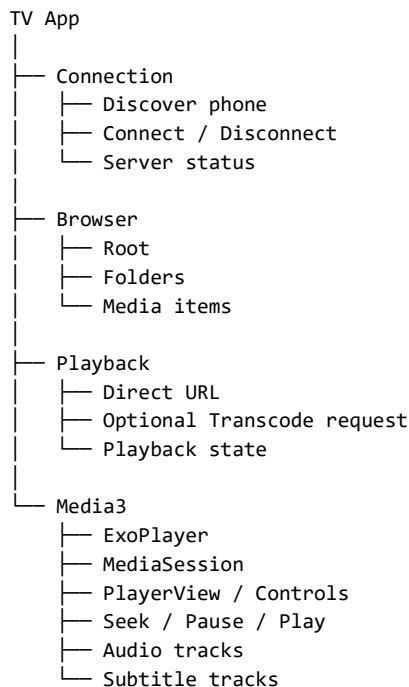
File on phone / USB
|



4. Mobile App Architecture



5. TV App Architecture



6. Server Protocol

Keep the protocol small. Example endpoints:

- GET /api/v1/info — server/device information
- GET /api/v1/files — list root or folder contents
- GET /api/v1/files/:id — metadata for one item
- GET /api/v1/stream/:id — direct media stream; supports HTTP Range
- POST /api/v1/play/:id — optional playback request with direct/transcode mode
- GET /api/v1/status — server/client status

Example playback command:

```
{  
  "fileId": "f_123",  
  "mode": "direct"  
}
```

or

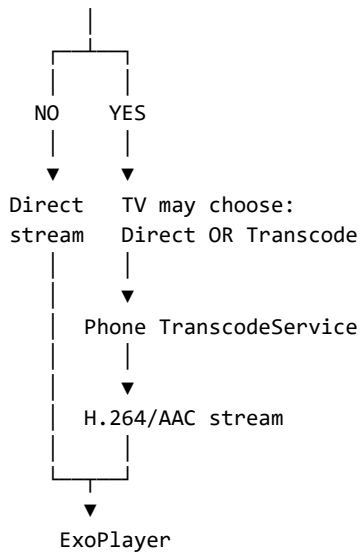
```
{  
  "fileId": "f_123",  
  "mode": "transcode"  
}
```

7. Transcoding Flow

TV selects a file



Is mobile transcoding enabled?



Transcoding is performed only on the phone. The TV sends a command/request; it never performs the conversion.

8. Settings Model

Server

- └ HTTP: ON
- └ HTTPS: OFF (optional)
- └ DLNA / UPnP: OFF (optional)
- └ Discovery: ON

Playback

- └ Transcoding: OFF by default

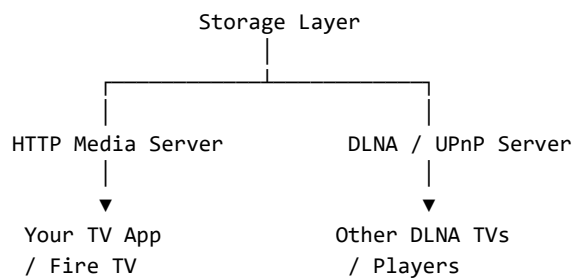
Security (future)

- └ Authentication: OFF by default

While server is running:

- └ Settings that affect server/transcoding are locked.
- └ Stop server → change settings → start server.

9. Direct vs DLNA Architecture



DLNA should be a compatibility feature, not the foundation of the application. The custom HTTP protocol gives full control over your dedicated TV client.

10. HTTPS

HTTPS should be optional. Plain HTTP is the simplest initial path for a trusted local network or phone hotspot. HTTPS can be added later for stronger transport security, but certificate handling should not delay the MVP.

11. Recommended Technology Stack

Area	Choice
Mobile	Kotlin, Jetpack Compose, Storage Access Framework, Foreground Service
TV	Kotlin, Jetpack Compose for TV, AndroidX Media3, ExoPlayer, MediaSession
Discovery	Android NSD / mDNS
Server	Lightweight embedded HTTP server; evaluate Ktor or NanoHTTPD
Transcoding	FFmpeg or another suitable Android-compatible media engine; add later
DLNA	Use/inspect a mature FOSS UPnP/DLNA implementation rather than implementing from scratch

12. Development Environment for 8 GB RAM / No dGPU

Avoid relying on an Android emulator. Develop with VS Code + command-line Android tooling and test on a real Android phone. Test the TV client on the actual Fire TV device. Android Studio can be used when needed, but it does not need to be your primary editor.

```
localstream/
├─ mobile/      Android phone application
├─ tv/          Android TV / Fire TV application
├─ protocol/    Shared API/protocol documentation
└─ docs/        Architecture and development notes
```

13. Build Roadmap

Phase 1 — Proof of concept: Android app → choose folder → HTTP server → stream one MP4 → Range requests

Phase 2 — TV playback: TV app → connect → browse → Media3/ExoPlayer → play video

Phase 3 — Real server: Foreground service → discovery → status → logs → multiple media types

Phase 4 — Storage: USB OTG → SD card → persistent folder permissions → large-file handling

Phase 5 — Transcoding: Global mobile setting → per-file TV option → transcoding endpoint → H.264/AAC

Phase 6 — Compatibility: DLNA → HTTPS → authentication → QR/manual pairing

14. Recommended MVP Boundary

Do not start with transcoding, DLNA, HTTPS, authentication, or advanced metadata. The first successful milestone should be:

```
Choose folder
↓
Start server
↓
Discover/connect from TV
```

↓
Browse files
↓
Play MP4/MKV directly
↓
Seek / pause / resume
↓
Stop server

Once this works reliably over a phone hotspot, the core architecture is proven. USB, transcoding, DLNA and HTTPS can then be added as independent modules.

15. Design Principles

- Direct streaming first; transcoding is optional.
- Phone owns storage and transcoding; TV owns playback.
- One StorageService feeds HTTP, transcoding, and DLNA.
- Use HTTP Range requests for seeking and large media.
- Keep DLNA as a compatibility layer.
- Keep HTTPS optional rather than making certificate management part of the MVP.
- Use Media3/ExoPlayer instead of building a player.
- Use a foreground service so the phone server survives leaving the app UI.
- Avoid an emulator on the 8 GB development machine; test on real devices.